Disaster Management Resource Management System

Algorithm Document | C++ Console Application

1. Objective

To help organize disaster-relief supplies and requests through a simple computer-based system. The program maintains available quantities of resources (water bottles, food packets, medicines, first-aid kits) and records every successful dispatch so that resources can be tracked during an emergency. A dispatch can also be reversed using the undo feature.

Concepts covered: structures, vectors, functions, linear search, menu-driven programming (do-while and switch), input validation and stack-based undo (LIFO).

2. Inputs, Outputs and Features

Item	Description
Input	Menu choice (1-8), resource name, quantity, request number
Output	Resource list, request list with status, activity history, success / error messages
Feature 1	Add Resource - add a new resource or increase stock of an existing one
Feature 2	Display Resources - show all resources and available quantities
Feature 3	Register Request - save a new relief request with status Pending
Feature 4	Display Requests - show all requests and their status
Feature 5	Dispatch - fulfil a pending request if stock is sufficient
Feature 6	Undo Last Dispatch - reverse the most recent successful dispatch
Feature 7	Display History - show all activities performed
Feature 8	Exit - close the application

3. Data Structures Used

Data structure	Purpose
struct Resource	Stores a resource name and its available quantity.
struct Request	Stores the requested resource name, quantity and status (dispatched: true / false).
struct DispatchRecord	Stores request index, resource index and quantity - enough information to undo the most recent dispatch.
vector<Resource> resources	Dynamic list of all resources in stock.
vector<Request> requests	Dynamic list of all registered requests.
vector<DispatchRecord> dispatchStack	Used as a **stack** (LIFO): push on dispatch, pop on undo.
vector<string> historyLog	Stores a text entry for every activity.

4. Main Algorithm

1. **Start** the program.
2. Create empty lists for resources, requests, dispatch stack and history.
3. Display the main menu.
4. Read the user's choice. If the input is not a number, clear the input, show a message and go to Step 3.
5. If the choice is **1**, ask for a resource name and quantity; add it or update its stock.
6. If the choice is **2**, display all available resources.
7. If the choice is **3**, ask for a resource name and required quantity; register a pending request.
8. If the choice is **4**, display all requests and their statuses.
9. If the choice is **5**, select a pending request, check the available stock, and dispatch the requested quantity if stock is sufficient.
10. If the choice is **6**, restore the stock and pending status of the most recent successful dispatch.
11. If the choice is **7**, display the activity history.
12. If the choice is **8**, print the exit message and terminate the loop.
13. For any other choice, display an error message.
14. Repeat from Step 3 until the user chooses Exit.
15. **Stop**.

5. Detailed Algorithms of Each Operation

5.1 Add Resource

1. Read the resource name and the quantity to add.
2. If the quantity is not a valid number, show 'Invalid quantity' and return.
3. If the name is empty or the quantity is 0 or negative, show an error and return.
4. Search the resource list for the name (findResource).
5. If not found, create a new resource with the given quantity; otherwise add the quantity to the existing stock.
6. Add 'Added X units of NAME' to the history and show a success message.

5.2 Register Request

1. Read the requested resource name and the quantity required.
2. Validate the input (valid number, name not empty, quantity greater than 0).
3. Create a new request with status **Pending** and append it to the request list.
4. Add the registration to the history and show a success message.

5.3 Dispatch Resource

1. If there are no requests, show 'No requests available' and return.
2. Display all requests and read the request number.
3. If the number is invalid or outside 1 to N, show an error and return.
4. If the request is already dispatched, show a message and return.
5. Find the requested resource in the inventory. If it is missing, show a message and return.
6. If available stock is less than the requested quantity, show 'Insufficient stock' - the request stays Pending - and return.
7. Otherwise: stock = stock - quantity, and mark the request as **Dispatched**.
8. Push a DispatchRecord (request index, resource index, quantity) onto the stack.
9. Add the dispatch to the history and show a success message.

5.4 Undo Last Dispatch

1. If the dispatch stack is empty, show 'No dispatch to undo' and return.
2. Pop the last DispatchRecord from the stack.
3. Add the quantity back to the resource stock.
4. Mark the corresponding request as **Pending** again.
5. Add the undo action to the history and show a success message.

5.5 Display Resources / Requests / History

1. If the list is empty, print a 'No ... available' message and return.
2. Otherwise loop through the list from the first item to the last and print each item with its details (requests show Pending / Dispatched).

6. Pseudocode of the Main Loop

```
START
  resources, requests, dispatchStack, historyLog = empty
  DO
    PRINT menu
    READ choice
    IF choice is not a number THEN show message; CONTINUE
    SWITCH choice
      1: addResource()          2: displayResources()
      3: registerRequest()      4: displayRequests()
      5: dispatchResource()     6: undoLastDispatch()
      7: displayHistory()       8: PRINT exit message
      default: PRINT invalid choice
    END SWITCH
  WHILE choice != 8
STOP
```

7. Time Complexity

Let **n** = number of resources, **r** = number of requests and **h** = number of history entries.

Operation	Time	Reason
Add Resource	O(n)	Linear search for the resource name
Display Resources	O(n)	Visits every resource once
Register Request	O(1) amortized	Appends to the end of a vector
Display Requests	O(r)	Visits every request once
Dispatch	O(r + n)	Displays requests, then searches the resource
Undo Last Dispatch	O(1)	Pops the top of the stack and updates by index
Display History	O(h)	Visits every history entry once

Space complexity is O(n + r + h) because all data is stored in memory in vectors.

8. Sample Run

```
Choice: 1
Resource name: WaterBottle
Quantity to add: 200
Resource added/updated successfully.

Choice: 3
Requested resource name: WaterBottle
Quantity required: 50
Request registered successfully.

Choice: 5
Request 1 | Resource: WaterBottle | Quantity: 50 | Status: Pending
Enter request number to dispatch: 1
Resource dispatched successfully.

Choice: 2
Name: WaterBottle | Available quantity: 150

Choice: 6
Last dispatch undone successfully. Stock restored.

Choice: 2
Name: WaterBottle | Available quantity: 200

Choice: 8
Exiting Disaster Management System.
```

9. Limitations and Future Improvements

- Data is stored in memory and is lost when the program closes.
- Resource and request names are case-sensitive (Water and water are different).
- Only a learning / demo project, not a production emergency-response system.
- Future versions can add file or database storage, dates and locations, priority levels, multiple relief centres, user login and reports.